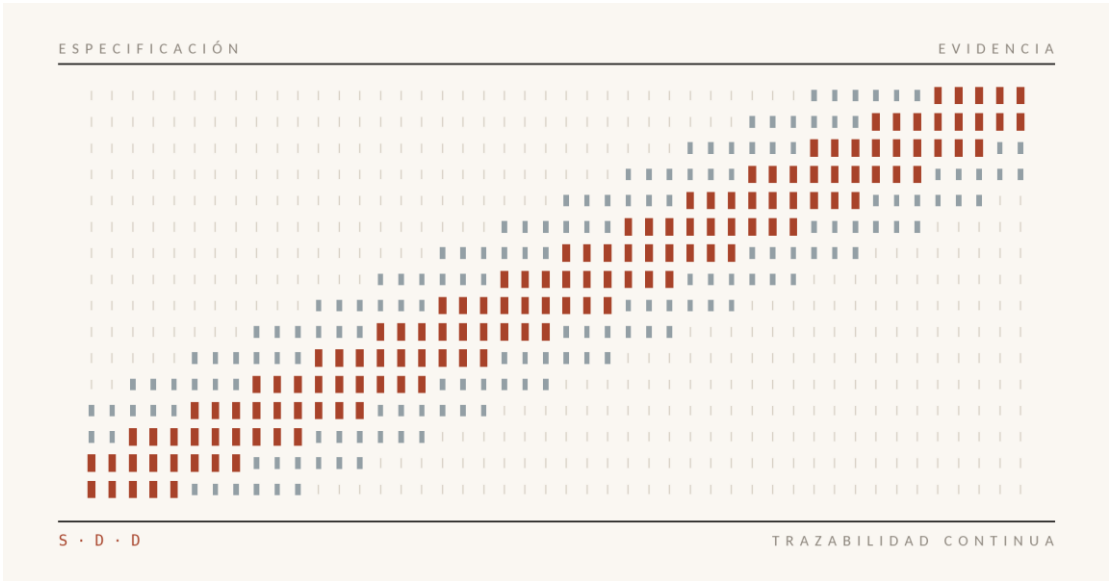


Specification-Driven Development (SDD)

Guía para aplicarlo en cualquier proyecto



LA REGLA CENTRAL

No implementar algo que no se entiende; no dar por aprobado algo que no se decidió; no declarar terminado algo que no se verificó.

Contenido

1	Qué es SDD y qué problema resuelve	3
2	El ciclo completo.....	4
3	Las etapas, una por una	4
3.1	REQUIREMENTS — Requisitos.....	4
3.2	SPECIFICATION — Especificación	5
3.3	ARCHITECTURE — Arquitectura	6
3.4	CONTRACTS — Contratos.....	7
3.5	INVARIANTS — Invariantes.....	7
3.6	EVALS / TESTS — Evaluaciones y pruebas	8
3.7	PLAN — Plan.....	8
3.8	TASKS — Tareas.....	8
3.9	IMPLEMENTATION — Implementación.....	9
3.10	VALIDATION — Validación	9
3.11	REVIEW — Revisión	10
3.12	DELIVERY — Entrega	10
4	La disciplina que hace que SDD funcione	11
4.1	Clasificá cada afirmación	11
4.2	Establecé una fuente de verdad.....	11
4.3	Usá identificadores estables.....	11
4.4	Mantené trazabilidad	12
5	Gestión de cambios: cómo no romper las reglas cuando el proyecto evoluciona	12
6	Cómo adaptar SDD a cualquier tipo de proyecto	13
7	Cómo manejar la incertidumbre sin bloquear el proyecto.....	14
8	Cómo trabajar con agentes de IA bajo SDD.....	15
9	Errores que tenés que evitar	16
10	Plantillas reutilizables para cualquier proyecto.....	16
10.1	Ficha inicial del proyecto.....	16
10.2	Registro de requisitos.....	18
10.3	Registro de decisiones.....	18
10.4	Ficha de tarea.....	19
10.5	Informe de entrega	20
11	Cómo saber si podés avanzar de una etapa a otra.....	21
12	Cómo aplicar SDD en el trabajo diario.....	22
13	La versión mínima y la versión rigurosa	23
14	Tu código mental permanente	23
15	Cómo aprenderlo de verdad.....	24

Specification-Driven Development (SDD): guía para aplicarlo en cualquier proyecto

El objetivo no es simplemente «trabajar con documentación antes de programar». Es construir un sistema de trabajo en el que cada decisión, tarea, implementación y validación tenga una razón explícita y evidencia verificable.

SDD puede aplicarse a software, productos digitales, automatizaciones, procesos operativos, análisis de datos, proyectos de ingeniería y otros trabajos complejos. Lo que cambia es el tipo de especificación, los riesgos y la forma de verificar el resultado.

La regla central es:

REGLA CENTRAL

No implementar algo que no se entiende; no dar por aprobado algo que no se decidió; no declarar terminado algo que no se verificó.

Y una precisión importante: SDD no significa que nunca puedas cambiar de idea. Significa que los cambios se hacen de forma explícita, trazable y controlada, no de manera accidental.

1 Qué es SDD y qué problema resuelve

Definición práctica

Specification-Driven Development es un enfoque de trabajo en el que la especificación define qué debe lograrse y bajo qué condiciones, y el desarrollo se deriva de ella.

La especificación no es un documento decorativo. Es una referencia operativa para:

- Definir el problema y el resultado esperado.
- Acordar requisitos y restricciones.
- Diseñar una solución.
- Establecer contratos e invariantes.
- Planificar tareas.
- Implementar.
- Verificar el cumplimiento.
- Gestionar cambios.
- Aceptar y entregar resultados.

Qué evita

Sin un proceso SDD	Con un proceso SDD
Se programa desde una idea ambigua.	Se aclara qué se necesita antes de implementar.
Se inventan reglas durante el desarrollo.	Las reglas se documentan y se aprueban.
El código termina definiendo el producto por accidente.	La especificación gobierna el comportamiento esperado.
Se confunde «compila» con «funciona».	Se valida contra criterios de aceptación.
Los agentes modifican más de lo solicitado.	El alcance y los límites quedan explícitos.
Los cambios rompen funciones sin advertencia.	Se analiza el impacto antes de cambiar.
Nadie sabe qué quedó pendiente.	Se registra el estado y la evidencia.

Lo que SDD no es

SDD no es:

- Escribir una especificación enorme antes de empezar.
- Prohibir la experimentación.

- Evitar todo cambio de alcance.
- Suponer que la documentación siempre está correcta.
- Generar código automáticamente a partir de una especificación.
- Reemplazar el criterio profesional.
- Garantizar que un proyecto tendrá éxito.

SDD reduce la ambigüedad y hace verificable el trabajo; no elimina la incertidumbre.

2 El ciclo completo

Este es el ciclo que podés usar como estructura general:

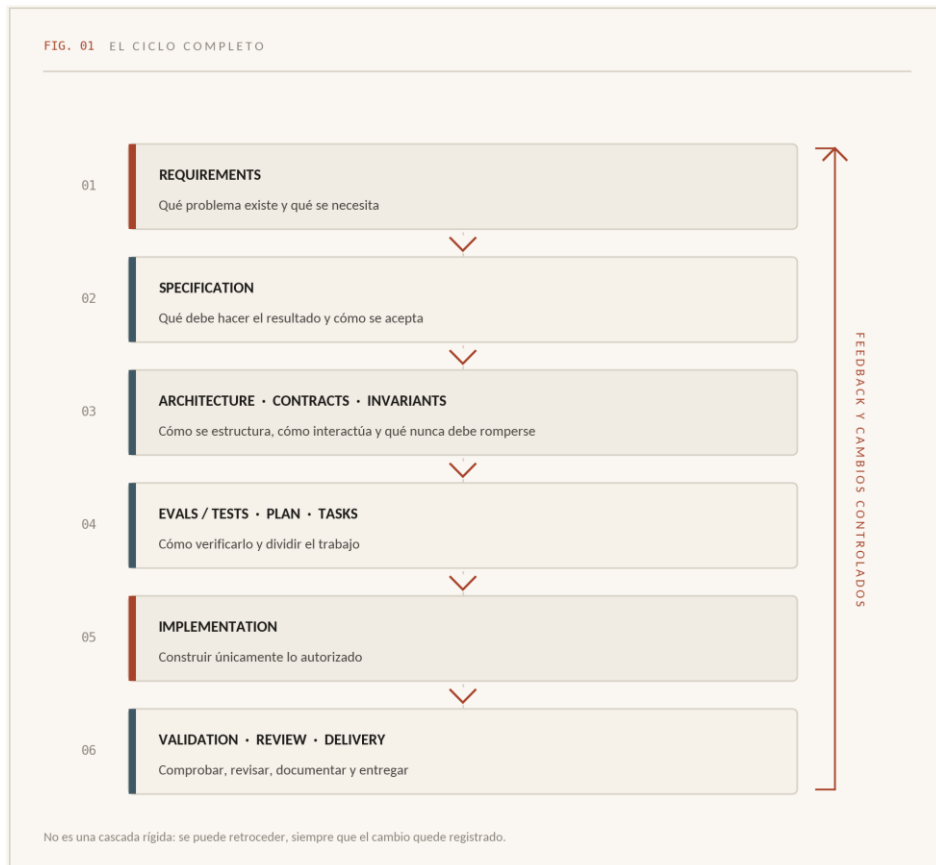


Fig. 01 — El ciclo completo: de la necesidad a la entrega, con retroalimentación controlada.

No es necesariamente una cascada rígida. En proyectos exploratorios podés volver de la especificación a los requisitos, o de la implementación a la arquitectura. La condición es registrar qué cambió, por qué y qué debe revisarse.

3 Las etapas, una por una

3.1 REQUIREMENTS — Requisitos

OBJETIVO

Entender el problema antes de elegir una solución.

Un requisito expresa una necesidad, capacidad, restricción o condición que el resultado debe cumplir.

PREGUNTAS OBLIGATORIAS

- ¿Qué problema se intenta resolver?
- ¿Quién tiene ese problema?
- ¿Quién utilizará o recibirá el resultado?
- ¿Qué resultado concreto se espera?

- ¿Qué está incluido?
- ¿Qué queda fuera?
- ¿Qué restricciones existen?
- ¿Qué riesgos o incertidumbres hay?
- ¿Cómo se sabrá que el problema quedó resuelto?

TIPOS DE REQUISITOS

Tipo	Qué describe	Ejemplo genérico
Funcional	Una capacidad o comportamiento	El usuario puede registrar una operación.
No funcional	Una propiedad de calidad	La respuesta debe cumplir un límite de latencia.
Restricción	Una condición obligatoria	El sistema debe utilizar una tecnología definida.
Regla de negocio	Una regla del dominio	Una operación no puede superar cierto límite autorizado.
Interfaz	Interacción entre sistemas o personas	Un servicio debe recibir un formato específico.
Seguridad	Protección y acceso	Solo roles autorizados pueden ejecutar una acción.
Operación	Instalación, soporte y continuidad	Debe existir un procedimiento de recuperación.

QUÉ PRODUCIR

- Objetivo y alcance.
- Actores o partes interesadas.
- Requisitos identificados.
- Restricciones.
- Supuestos.
- Riesgos.
- Preguntas abiertas.
- Prioridades.

REGLA

No convertir una suposición en requisito aprobado.

Si falta información, marcarla como pendiente. No completar el vacío con una decisión inventada.

3.2 SPECIFICATION — Especificación

OBJETIVO

Convertir las necesidades en una descripción clara, precisa y verificable del resultado esperado.

Un requisito puede ser correcto pero demasiado ambiguo para implementarlo.

Por ejemplo:

«El sistema debe ser rápido».

Eso no permite determinar qué significa cumplirlo.

Una especificación más verificable definiría qué operación se mide, bajo qué condiciones y cuál es el umbral aceptable. El umbral concreto debe surgir de una decisión o evidencia válida, no de una invención del desarrollador.

UNA ESPECIFICACIÓN DEBE ACLARAR

- Comportamiento esperado.
- Entradas y salidas.
- Estados y transiciones.
- Reglas y excepciones.
- Permisos.

- Errores posibles.
- Restricciones.
- Criterios de aceptación.
- Casos límite.
- Aspectos que todavía no están definidos.

CRITERIOS DE ACEPTACIÓN

Cada capacidad relevante debe tener condiciones que permitan determinar si cumple.

Ejemplo abstracto:

Dado un usuario autorizado.
Cuando ejecuta una operación con datos válidos.
Entonces el sistema registra el resultado esperado.
Y conserva las restricciones de integridad definidas.

Para un proceso no informático, el criterio puede ser un documento aprobado, una medición, una inspección o una prueba de campo.

REGLA

Si no se puede explicar cómo verificar un requisito, probablemente todavía está incompleto.

3.3 ARCHITECTURE — Arquitectura

OBJETIVO

Definir la estructura de la solución y las responsabilidades de sus partes.

La arquitectura responde cómo se organizará la solución, pero no debería inventar reglas de negocio que todavía no fueron aprobadas.

PARA CADA COMPONENTE, DEFINIR

- Nombre.
- Responsabilidad.
- Qué hace.
- Qué no hace.
- Dependencias.
- Entradas y salidas.
- Límites.
- Riesgos.
- Requisitos que satisface.
- Decisiones que lo justifican.

EJEMPLO DE FICHA

Campo	Contenido
Componente	Servicio de procesamiento
Responsabilidad	Procesar una solicitud validada
Dependencias	Repositorio de datos y política de autorización
Entradas	Solicitud validada
Salidas	Resultado o error definido
No debe hacer	Cambiar reglas de negocio por cuenta propia
Verificación	Pruebas de contrato y de comportamiento

REGLA

La arquitectura debe derivarse de los requisitos y las restricciones, no de preferencias tecnológicas aisladas.

Una tecnología puede ser una restricción aprobada, una decisión pendiente o una propuesta. Hay que etiquetarla correctamente.

3.4 CONTRACTS — Contratos

OBJETIVO

Definir con precisión cómo se relacionan los componentes, sistemas, equipos o personas.

Un contrato establece qué puede esperar cada parte de otra.

UN CONTRATO PUEDE ESPECIFICAR

- Formato de entrada.
- Formato de salida.
- Precondiciones.
- Postcondiciones.
- Errores posibles.
- Responsabilidad de cada parte.
- Límites de autorización.
- Compatibilidad.
- Comportamiento ante fallos.

En software, puede ser una API, un esquema de datos, una interfaz o un evento. En otros proyectos, puede ser un protocolo operativo, un formato de entrega o un acuerdo de intercambio.

REGLA

No alcanza con que dos componentes “se entiendan”; la interacción debe estar definida y ser verificable.

3.5 INVARIANTS — Invariantes

OBJETIVO

Identificar condiciones que deben mantenerse verdaderas durante toda la vida del sistema o proceso.

Un requisito describe algo que debe cumplirse. Un invariante expresa una propiedad que no debe violarse, incluso durante errores, cambios o situaciones excepcionales.

Ejemplos genéricos:

- Una entidad no puede pertenecer simultáneamente a dos propietarios incompatibles.
- Una operación confirmada no puede desaparecer sin dejar rastro.
- Un saldo debe conservar una relación definida con sus movimientos.
- Un proceso no puede avanzar si falta una aprobación obligatoria.

Los ejemplos son ilustrativos: cada proyecto debe definir sus propios invariantes.

PARA CADA INVARIANTE

- Expresar la condición.
- Explicar por qué importa.
- Identificar qué componentes pueden afectarla.
- Definir cómo se protege.
- Definir cómo se prueba.
- Definir cómo se detecta una violación.

REGLA

Los invariantes deben protegerse en el diseño y verificarse en las pruebas; no basta con escribirlos.

3.6 EVALS / TESTS — Evaluaciones y pruebas

OBJETIVO

Definir cómo se demostrará que el resultado satisface la especificación.

La verificación debe diseñarse antes de terminar la implementación, no improvisarse al final.

TIPOS DE VERIFICACIÓN

Tipo	Qué verifica
Inspección	Documentos, diseños, cálculos o código
Prueba unitaria	Comportamiento aislado
Prueba de integración	Interacción entre componentes
Prueba de contrato	Cumplimiento de interfaces
Prueba de aceptación	Cumplimiento de necesidades del usuario
Prueba de seguridad	Controles y amenazas relevantes
Prueba de rendimiento	Comportamiento bajo carga o condiciones definidas
Validación de campo	Funcionamiento en el entorno real
Revisión independiente	Evidencia y decisiones por una persona distinta

No todos los proyectos necesitan todos estos tipos. La estrategia debe ser proporcional al riesgo, el costo y el impacto de fallar.

UNA PRUEBA DEBE INDICAR

- Requisito o invariante relacionado.
- Precondiciones.
- Datos o materiales de prueba.
- Acción.
- Resultado esperado.
- Resultado observado.
- Estado: aprobada, fallida, bloqueada o no ejecutada.
- Evidencia.

REGLA

Una prueba definida no es una prueba ejecutada. Una prueba ejecutada no es necesariamente una prueba aprobada.

3.7 PLAN — Plan

OBJETIVO

Definir el orden lógico del trabajo y las dependencias.

El plan debe responder:

- ¿Qué hay que construir o producir?
- ¿En qué orden?
- ¿Qué depende de qué?
- ¿Qué riesgos bloquean el avance?
- ¿Qué resultado verificable entrega cada etapa?
- ¿Qué condiciones habilitan la siguiente etapa?

El plan no debe convertirse en una promesa falsa de certeza. Puede actualizarse cuando aparece evidencia nueva.

3.8 TASKS — Tareas

OBJETIVO

Convertir el plan en unidades de trabajo concretas, limitadas y verificables.

Una buena tarea no es «hacer el módulo» o «arreglar el sistema».

Una tarea debe dejar claro:

- Qué se modifica.
- Por qué.
- Qué requisito satisface.
- Qué queda fuera.
- Qué archivos, componentes o entregables pueden afectarse.
- Qué dependencias tiene.
- Qué pruebas deben ejecutarse.
- Qué evidencia se espera.
- Qué condición permite cerrarla.

REGLA

Una tarea no se cierra porque alguien diga que terminó. Se cierra cuando cumple sus criterios y presenta la evidencia acordada.

3.9 IMPLEMENTATION — Implementación

OBJETIVO

Construir lo definido, dentro del alcance autorizado.

Durante la implementación:

- No inventar reglas de negocio.
- No modificar requisitos silenciosamente.
- No ampliar el alcance por conveniencia.
- No eliminar trabajo existente sin autorización.
- No reemplazar componentes por duplicación aparente.
- No ocultar errores.
- No declarar funcionalidad terminada por tener una interfaz o estructura vacía.
- No confundir código generado con comportamiento validado.

Si aparece una ambigüedad relevante, se detiene la parte afectada y se solicita una decisión.

REGLA

Implementar es ejecutar una especificación, no reescribirla unilateralmente.

3.10 VALIDATION — Validación

OBJETIVO

Comprobar el resultado real contra los criterios definidos.

Hay que distinguir:

- **Implementado:** el cambio fue realizado.
- **Verificado por código:** se inspeccionó el código y se encontró la estructura esperada.
- **Verificado por test:** una prueba se ejecutó y produjo el resultado esperado.
- **Verificado por ejecución:** el sistema o proceso se ejecutó en condiciones definidas.
- **Validado por aceptación:** se comprobó que satisface la necesidad del usuario o interesado.



Fig. 02 — Estas categorías no son equivalentes: cada escalón exige una evidencia distinta.

Estas categorías no son equivalentes.

Un build exitoso no demuestra que las reglas de negocio sean correctas. Una prueba unitaria no demuestra que todo el sistema funcione. Una pantalla visible no demuestra que esté conectada a datos reales.

REGLA

La afirmación de éxito nunca puede ser más fuerte que la evidencia disponible.

3.11 REVIEW — Revisión

OBJETIVO

Evaluar si el resultado es correcto, coherente, seguro, mantenible y trazable.

La revisión debe comprobar:

- Cumplimiento del alcance.
- Cumplimiento de requisitos.
- Respeto por contratos e invariantes.
- Calidad de implementación.
- Pruebas y resultados.
- Riesgos residuales.
- Cambios no autorizados.
- Documentación actualizada.
- Pendientes y limitaciones.

REGLA

La revisión no debe limitarse a preguntar “¿funciona?”. También debe preguntar “¿es lo que se aprobó?”

3.12 DELIVERY — Entrega

OBJETIVO

Entregar el resultado junto con la evidencia necesaria para entenderlo y mantenerlo.

Una entrega debería incluir, según el proyecto:

- Qué se entregó.
- Qué requisitos satisface.
- Qué decisiones se aplicaron.
- Qué se modificó.
- Qué pruebas se ejecutaron.

- Qué resultados dieron.
- Qué no se probó.
- Qué limitaciones quedan.
- Qué riesgos permanecen.
- Qué pasos siguen.
- Qué versión o estado se está entregando.

REGLA

Una entrega completa incluye el producto y la evidencia de su estado.

4 La disciplina que hace que SDD funcione

Conocer las etapas no alcanza. Lo que vuelve consistente al método es el control de la información y de las decisiones.

4.1 Clasificá cada afirmación

En cualquier proyecto, separá explícitamente estas categorías:

Estado	Significado
Requisito aprobado	Necesidad aceptada como parte del alcance.
Decisión aprobada	Alternativa elegida por quien tiene autoridad para decidir.
Decisión pendiente	Todavía falta una definición.
Hipótesis	Suposición que requiere validación.
Propuesta	Alternativa sugerida, todavía no aprobada.
Implementación existente	Algo que está presente en el producto o proceso.
Implementación verificada	Algo cuya existencia o comportamiento fue comprobado con evidencia.
Fuera de alcance	Algo que explícitamente no se hará en este incremento o proyecto.

ATENCIÓN

No uses «decidido», «aprobado», «implementado» y «verificado» como sinónimos.

4.2 Establecé una fuente de verdad

Cada proyecto necesita una referencia clara para saber qué está vigente.

Una estructura posible:

1. Mandatos, obligaciones y restricciones superiores.
2. Especificación vigente y decisiones aprobadas.
3. Contratos, arquitectura y criterios de aceptación.
4. Estado real del producto o proceso y evidencia.
5. Conversaciones, notas y contexto auxiliar.
6. Hipótesis y propuestas nuevas.

El orden exacto depende del proyecto. Lo importante es definirlo y evitar que una conversación casual sobrescriba una decisión formal.

La documentación también puede estar equivocada o desactualizada. Si contradice la realidad, no ocultes la contradicción: registrala y resóvela.

4.3 Usá identificadores estables

Asigná identificadores a los elementos importantes:

Requisitos	REQ-001
Decisiones	DEC-001
Componentes	CMP-001
Contratos	CTR-001
Invariantes	INV-001
Pruebas	TST-001
Tareas	TASK-001
Riesgos	RSK-001

Son ejemplos de nomenclatura, no una norma universal. Podés usar otra convención, siempre que sea consistente.

El propósito es poder decir:

Esta tarea implementa REQ-014, aplica DEC-006, afecta CMP-003 y se verifica con TST-021 y TST-022.

Así se evita que los documentos sean colecciones de texto sin relación entre sí.

4.4 Mantén trazabilidad

La trazabilidad conecta la necesidad con el resultado:



Fig. 03 — Requisito → Especificación y decisión → Componente y contrato → Tarea e implementación → Prueba y evidencia → Aceptación y entrega.

La trazabilidad debe permitir recorrer el camino en ambos sentidos:

- Desde el requisito: ¿dónde se implementa y cómo se verifica?
- Desde un cambio: ¿qué requisito o decisión lo justifica?
- Desde una prueba fallida: ¿qué comportamiento o requisito está comprometido?
- Desde un requisito eliminado: ¿qué componentes, pruebas y documentos deben revisarse?

5 Gestión de cambios: cómo no romper las reglas cuando el proyecto evoluciona

Un proyecto real cambia. La disciplina SDD no consiste en congelarlo, sino en impedir cambios invisibles.

Proceso de cambio

1. Registrar el cambio solicitado.
2. Identificar el problema que intenta resolver.
3. Determinar si es corrección, mejora, cambio de alcance o nueva necesidad.
4. Analizar requisitos, decisiones, arquitectura, contratos, invariantes y pruebas afectados.

5. Presentar alternativas y consecuencias.
6. Obtener aprobación cuando corresponda.
7. Actualizar la especificación y los documentos afectados.
8. Crear o ajustar tareas.
9. Implementar el cambio.
10. Ejecutar las verificaciones afectadas.
11. Registrar el resultado y la decisión final.

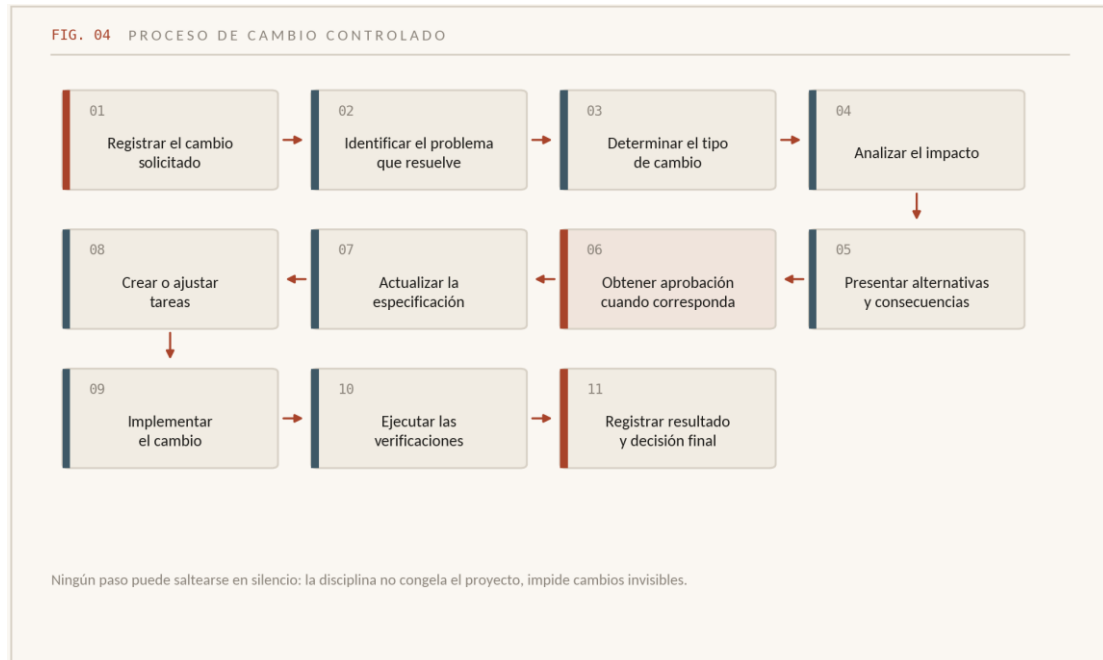


Fig. 04 — Los once pasos del cambio controlado, en secuencia.

Ejemplo

Se solicita agregar una nueva capacidad.

No conviene empezar directamente a programarla. Primero hay que establecer:

- ¿Es parte del alcance aprobado?
- ¿Qué necesidad satisface?
- ¿Qué comportamiento nuevo introduce?
- ¿Qué reglas existentes modifica?
- ¿Qué componentes afecta?
- ¿Qué pruebas nuevas requiere?
- ¿Qué riesgos aparecen?
- ¿Quién debe aprobarla?

Si la respuesta todavía no está definida, el estado correcto es decisión pendiente, no «lo resolvemos mientras programamos».

6 Cómo adaptar SDD a cualquier tipo de proyecto

El ciclo general es el mismo. Lo que cambia es qué se considera una especificación y qué evidencia es suficiente.

Tipo de proyecto	Especificación	Validación típica
Software	Requisitos funcionales, arquitectura, contratos, comportamiento	Tests, ejecución, revisión, aceptación
Análisis de datos	Preguntas, fuentes, definiciones, transformaciones, reglas de calidad	Reconciliación, controles de datos, reproducibilidad

Tipo de proyecto	Especificación	Validación típica
Automatización	Disparadores, entradas, acciones, excepciones, permisos	Pruebas de escenarios, logs, ejecución controlada
Ingeniería física	Requisitos, planos, materiales, tolerancias, normas aplicables	Cálculos, inspecciones, ensayos, certificación
Operaciones	Procedimientos, roles, entradas, salidas, niveles de servicio	Auditorías, mediciones, simulaciones, resultados operativos
Investigación	Pregunta, hipótesis, método, datos, criterios de evaluación	Experimentos, análisis, replicación, revisión
Producto o negocio	Problema, usuarios, propuesta, alcance, métricas y restricciones	Investigación con usuarios, experimentos, resultados medidos
Proyecto documental	Audiencia, propósito, estructura, requisitos de contenido	Revisión contra criterios, aprobación del destinatario

La diferencia esencial

- En software, una prueba puede ser automatizada y repetible.
- En una obra, la verificación puede requerir mediciones, inspecciones y profesionales habilitados.
- En investigación, el resultado puede ser incierto y no estar garantizado de antemano. La especificación debe definir el método y los criterios de evaluación, no inventar el resultado esperado.
- En un proyecto de negocio, un supuesto comercial no se vuelve verdadero porque esté escrito en el plan. Se debe validar mediante evidencia.

SDD no significa exigir certeza donde no puede haberla. Significa especificar qué se sabe, qué se supone y cómo se investigará lo desconocido.

7 Cómo manejar la incertidumbre sin bloquear el proyecto

Uno de los errores más comunes es creer que no se puede avanzar hasta tener todo definido.

Eso no es necesario. Lo importante es diferenciar qué incertidumbre bloquea qué trabajo.

Clasificación útil

- **Bloqueante:** sin esta definición, implementar podría causar un error relevante.
- **No bloqueante:** se puede avanzar en otras tareas sin asumir esa definición.
- **Exploratoria:** requiere investigación, prototipo, experimento o consulta.
- **Aceptada temporalmente:** se permite avanzar con una hipótesis explícita, limitada y revisable.



Fig. 05 — Qué habilita cada tipo de incertidumbre.

Cada hipótesis aceptada temporalmente debería indicar:

- Qué se está suponiendo.
- Por qué se acepta.
- Qué parte del proyecto afecta.
- Qué riesgo introduce.
- Cómo se validará.
- Quién debe revisarla.
- Cuándo o bajo qué condición vence.

ATENCIÓN

No conviertas una hipótesis temporal en una regla permanente por el simple hecho de que el código ya la utiliza.

8 Cómo trabajar con agentes de IA bajo SDD

Si usas Claude Code, Codex, OpenCode u otro agente, trata al agente como un ejecutor técnico que necesita instrucciones, límites y criterios de verificación.

No le delegates decisiones de producto de forma implícita.

Antes de darle una tarea

El agente debe conocer:

- La fuente de verdad vigente.
- Los requisitos y decisiones relacionados.
- El alcance exacto.
- Los componentes autorizados.
- Las restricciones.
- Las pruebas requeridas.
- La evidencia que debe entregar.
- Qué hacer si encuentra contradicciones o información faltante.

Durante la tarea

Exigí que el agente:

- Inspeccione el estado real antes de modificar.
- Distinga lo existente de lo supuesto.
- No invente requisitos.
- No cambie archivos fuera del alcance.
- No sobrescriba trabajo existente sin autorización.
- No elimine componentes por parecer duplicados.
- No declare tests ejecutados si no los ejecutó.
- No declare éxito si las pruebas fallan.
- Detenga la parte afectada si falta una decisión relevante.
- Informe los cambios y los pendientes.

Al terminar

El agente debe informar:

- Qué cambió.
- Por qué lo cambió.
- Qué requisitos satisface.
- Qué archivos o componentes afectó.
- Qué comandos o verificaciones ejecutó.
- Resultados exactos.

- Qué no pudo verificar.
- Qué riesgos o pendientes detectó.
- Si modificó algo fuera del alcance y por qué.

Regla de oro con agentes

REGLA DE ORO

No delegues una tarea ambigua esperando que el agente adivine correctamente.

La IA puede proponer alternativas, detectar contradicciones y ayudar a escribir especificaciones. Pero una propuesta del agente no se convierte automáticamente en una decisión aprobada.

9 Errores que tenés que evitar

1. Programar antes de entender

Si la necesidad, las reglas o los límites no están claros, primero hay que analizarlos o aislar explícitamente la incertidumbre.

2. Confundir propuesta con decisión

Una recomendación técnica no está aprobada hasta que quien corresponde la acepta.

3. Especificar demasiado pronto detalles inciertos

No agregues precisión ficticia. Marcá lo desconocido y definí cómo obtener evidencia.

4. Documentar sin mantener

Una especificación desactualizada puede ser más peligrosa que una especificación incompleta claramente identificada.

5. Hacer tareas gigantes

Dividí el trabajo en incrementos pequeños que produzcan resultados verificables.

6. Probar solo el camino feliz

Incluí errores, permisos, límites, datos inválidos, fallos de dependencias y condiciones excepcionales relevantes.

7. Medir progreso por cantidad de código o documentos

Medilo por requisitos satisfechos, evidencia obtenida y riesgos reducidos.

8. Tratar el proceso como burocracia

La documentación debe servir para decidir, implementar, verificar y mantener. Si no cumple una función, simplificala.

10 Plantillas reutilizables para cualquier proyecto

No necesitás crear decenas de documentos desde el primer día. Empezá con un conjunto pequeño y agregá estructura cuando el riesgo o la complejidad lo justifiquen.

10.1 Ficha inicial del proyecto

FICHA DEL PROYECTO

1. Identificación

Nombre:

Responsable:

Versión:

Estado:

Fecha de actualización:

2. Problema

¿Qué problema concreto se intenta resolver?

3. Objetivo

¿Qué resultado debe lograrse?

4. Usuarios o destinatarios

¿Quién utilizará, recibirá o será afectado por el resultado?

5. Alcance

Incluido	Excluido

6. Restricciones

Técnicas:

Económicas:

Legales o regulatorias:

Operativas:

Temporales:

7. Criterios generales de éxito

¿Cómo se determinará que el proyecto cumplió su objetivo?

8. Incertidumbres

Preguntas abiertas:

Hipótesis:

Riesgos:

Decisiones pendientes:

9. Fuente de verdad

¿Qué documentos y decisiones gobiernan el proyecto?

10. Aprobación

¿Quién puede aprobar requisitos, cambios y entregas?

10.2 Registro de requisitos

REGISTRO DE REQUISITOS

ID	Requisito	Tipo	Prioridad	Estado	Criterio de aceptación	Verificación
REQ-001				Pendiente		

Estados posibles: propuesto, aprobado, en desarrollo, implementado, verificado, bloqueado, rechazado, reemplazado.

10.3 Registro de decisiones

REGISTRO DE DECISIONES

DEC-XXX — [Título]

Estado:

Fecha:

Responsable de la decisión:

Problema que resuelve:

Contexto:

Requisitos relacionados:

Estado: Propuesta / Aprobada / Rechazada / Reemplazada

ALTERNATIVAS

DECISIÓN

¿Qué alternativa se eligió?

JUSTIFICACIÓN

¿Por qué?

CONSECUENCIAS

Beneficios:

Costos:

Riesgos:

Dependencias:

Impacto sobre decisiones existentes:

EVIDENCIA O FUNDAMENTO

¿Qué datos, restricciones o criterios respaldan la decisión?

REVISIÓN

¿Qué condición podría justificar revisarla?

10.4 Ficha de tarea

TASK-XXX — [NOMBRE DE LA TAREA]

ESTADO

Pendiente / En curso / Bloqueada / Implementada / Verificada / Cerrada

OBJETIVO

¿Qué resultado concreto debe producir?

TRAZABILIDAD

Requisitos:

Decisiones:

Componentes:

Contratos:

Invariantes:

ALCANCE

Incluye

No incluye

ESTADO INICIAL

¿Qué existe y qué se verificó antes de empezar?

CAMBIOS AUTORIZADOS

DEPENDENCIAS Y RIESGOS

CRITERIOS DE ACEPTACIÓN

VALIDACIÓN

Comandos, pruebas, inspecciones o mediciones:

Resultado esperado:

Evidencia requerida:

RESULTADO FINAL

Cambios realizados:

Verificaciones ejecutadas:

Resultado:

Pendientes:

Desviaciones:

Evidencia:

10.5 Informe de entrega

INFORME DE ENTREGA

1. Resultado

¿Qué se entrega?

2. Alcance realizado

¿Qué quedó efectivamente implementado o producido?

3. Trazabilidad

¿Qué requisitos y decisiones satisface?

4. Validaciones

Verificación	Estado	Evidencia

5. Limitaciones

¿Qué no se implementó, probó o pudo verificar?

6. Riesgos residuales

¿Qué sigue siendo incierto o requiere seguimiento?

7. Desviaciones

¿Hubo cambios fuera del alcance aprobado?

8. Pendientes

¿Qué queda por hacer?

9. Estado de aceptación

Pendiente / Aceptado / Aceptado con observaciones / Rechazado

10. Próximo paso

¿Qué debe suceder a continuación?

11 Cómo saber si podés avanzar de una etapa a otra

No necesitás una ceremonia formal para cada paso, pero sí una condición de salida.

Checklist de control de avance

0/10

Marcá una condición cuando esté satisfecha con evidencia suficiente. No es necesario que todas se resuelvan de la misma manera en proyectos de distinto riesgo.

Transición	Condición de salida
☐ Requirements → Specification	El problema, los destinatarios, el alcance y las restricciones están suficientemente entendidos. Las incertidumbres están registradas.
☐ Specification → Architecture	Los requisitos relevantes tienen un comportamiento esperado y criterios de aceptación, o están explícitamente marcados como pendientes.
☐ Architecture → Contracts	Las responsabilidades, dependencias y límites principales están definidos.
☐ Contracts → Tests	Las interacciones relevantes, errores y condiciones críticas pueden verificarse.
☐ Tests → Plan	Existe una estrategia de verificación proporcional al riesgo y un orden de trabajo viable.
☐ Plan → Tasks	El trabajo está dividido en tareas con alcance, dependencias y criterios de cierre.
☐ Tasks → Implementation	La tarea está autorizada, las dependencias están disponibles y no quedan ambigüedades bloqueantes.
☐ Implementation → Validation	El cambio está identificado, el alcance fue respetado y hay evidencia de lo que se realizó.

Transición	Condición de salida
☐ Validation → Review	Los resultados de las pruebas y las limitaciones están registrados.
☐ Review → Delivery	El resultado fue revisado, los pendientes están explícitos y la aceptación corresponde a quien tiene autoridad.

Reiniciar checklist

FIG. 06 CONDICIONES DE SALIDA ENTRE ETAPAS

01	Requirements	→ Specification	Problema, destinatarios, alcance y restricciones entendidos
02	Specification	→ Architecture	Comportamiento esperado y criterios de aceptación definidos
03	Architecture	→ Contracts	Responsabilidades, dependencias y límites definidos
04	Contracts	→ Tests	Interacciones, errores y condiciones críticas verificables
05	Tests	→ Plan	Estrategia de verificación proporcional al riesgo
06	Plan	→ Tasks	Trabajo dividido con alcance y criterios de cierre
07	Tasks	→ Implementation	Tarea autorizada y sin ambigüedades bloqueantes
08	Implementation	→ Validation	Alcance respetado y evidencia de lo realizado
09	Validation	→ Review	Resultados de pruebas y limitaciones registrados
10	Review	→ Delivery	Revisado, pendientes explícitos y aceptación válida

Si una condición no se cumple: volver a analizar, reducir el alcance o abrir una tarea exploratoria.

Fig. 06 — Las diez condiciones de salida, de Requirements a Delivery.

IMPORTANTE

No uses este checklist como una máquina que obliga a avanzar. Si una condición no se cumple, podés volver a analizar, reducir el alcance o realizar una tarea exploratoria.

12 Cómo aplicar SDD en el trabajo diario

La práctica cotidiana puede ser muy simple.

Antes de empezar una sesión

- Revisá la especificación vigente.
- Revisá las decisiones relevantes.
- Confirmá el estado real del proyecto.
- Elegí una tarea autorizada.
- Identificá el resultado esperado y cómo lo vas a verificar.

Durante el trabajo

- No amplíes el alcance por iniciativa propia.
- Registrá las decisiones que afecten el resultado.
- Si aparece una contradicción, detené la parte afectada.
- Si surge una hipótesis, etiquetala como tal.
- Conservá evidencia de las verificaciones.

Al terminar

- Compará el resultado con el alcance de la tarea.
- Ejecutá las validaciones correspondientes.
- Registrá los resultados reales.
- Actualizá el estado de la tarea.
- Documentá pendientes y riesgos.
- Definí el siguiente paso autorizado.

Regla para cuando no sabés qué hacer

Hacete estas preguntas, en este orden:

1. ¿Qué estoy intentando satisfacer?
2. ¿Qué documento o decisión lo respalda?
3. ¿Qué información falta?
4. ¿Puedo avanzar sin asumir algo importante?
5. ¿Qué evidencia necesito para decir que terminé?

Si no podés responder las preguntas 1 o 2, probablemente falta trazabilidad. Si no podés responder la 3, puede haber una ambigüedad oculta. Si no podés responder la 5, la tarea todavía no tiene una definición suficiente de terminado.

13 La versión mínima y la versión rigurosa

SDD debe adaptarse al riesgo, no al revés.

Proyecto pequeño y de bajo riesgo	Proyecto complejo o de alto riesgo
Ficha de objetivo y alcance	Especificación formal y controlada
Lista breve de requisitos	Requisitos con IDs y trazabilidad
Decisiones en un registro simple	Registro formal de decisiones y aprobaciones
Criterios de aceptación concisos	Contratos, invariantes y análisis de riesgos
Pruebas manuales o automatizadas básicas	Estrategia de verificación integral
Revisión directa	Revisión independiente cuando corresponda
Entrega breve	Informe de validación, limitaciones y aceptación

Un trabajo pequeño no necesita una burocracia enorme. Pero incluso un cambio pequeño debería tener un objetivo, un alcance y una forma de comprobar que quedó bien.

En proyectos regulados, críticos o de seguridad, pueden ser necesarias normas, controles, aprobaciones y documentación adicionales. SDD no reemplaza esas obligaciones.

14 Tu código mental permanente

Estas son las reglas que conviene interiorizar:

01 La necesidad precede a la solución.	10 Las tareas deben tener alcance y criterios de cierre.
02 La especificación gobierna el resultado esperado.	11 La implementación no puede cambiar unilateralmente la especificación.
03 Las decisiones deben ser explícitas.	12 Los cambios requieren análisis de impacto.
04 Las hipótesis no son hechos.	13 La evidencia determina qué se puede afirmar.

05 Las propuestas no son aprobaciones.	14 Lo que no se verificó debe declararse como no verificado.
06 La arquitectura debe responder a requisitos y restricciones.	15 La entrega incluye limitaciones y pendientes.
07 Los contratos definen las interacciones.	16 La documentación debe mantenerse coherente con la realidad.
08 Los invariantes protegen propiedades críticas.	17 La incertidumbre se registra y se gestiona, no se oculta.
09 Las pruebas deben derivarse de los requisitos.	18 El rigor debe ser proporcional al riesgo.

15 Cómo aprenderlo de verdad

Leer la metodología es el primer paso. Aprenderla consiste en practicarla sobre proyectos reales.

Te propongo esta secuencia de aprendizaje:

Nivel 1 — Comprensión

Aprender a distinguir requisitos, decisiones, hipótesis, propuestas y evidencia.

Nivel 2 — Especificación

Tomar una idea sencilla y transformarla en requisitos verificables, alcance y criterios de aceptación.

Nivel 3 — Diseño

Derivar arquitectura, contratos e invariantes sin inventar reglas que no fueron definidas.

Nivel 4 — Planificación

Dividir el trabajo en tareas pequeñas, trazables y verificables.

Nivel 5 — Ejecución

Implementar o producir el resultado siguiendo el alcance autorizado.

Nivel 6 — Auditoría

Revisar un proyecto terminado y detectar afirmaciones sin evidencia, requisitos sin pruebas, decisiones implícitas y documentación desactualizada.

Nivel 7 — Adaptación

Aplicar el método a software, operaciones, datos, investigación y proyectos físicos, ajustando los controles al riesgo.



Fig. 07 — La secuencia completa, del criterio básico a la adaptación por riesgo.

La regla final

LA REGLA FINAL

No busques «cumplir el proceso» de manera mecánica. Busca que ninguna parte importante del trabajo quede sin fundamento, sin dueño, sin criterio de aceptación o sin evidencia.

Ese es el hábito que permite aplicar SDD de manera consistente a proyectos distintos y mantenerlo incluso cuando cambian los requisitos, las personas, las herramientas o la tecnología.